

AI-Assisted Crypto Portfolio Analytics

Senior Full-Stack Engineer Assessment

Project Specification

Overview

Design, build, and deploy a crypto portfolio analytics dashboard.

This assignment evaluates your ability to build a full-stack application, implement financial calculations correctly, work effectively with an AI coding agent, and deliver a production-ready result.

AI coding-agent use is expected. Your workflow is part of the assessment. You remain responsible for every design decision, calculation, and line of submitted code.

Project Summary

A crypto investor trades several assets across two exchanges. Build an application that imports the supplied trade history and shows the current state and performance of the portfolio.

The investor should be able to answer:

- What assets do I hold?
- What is each position worth now?
- What is my average cost and remaining cost basis?
- How much profit or loss have I realized?
- How much profit or loss is still unrealized?
- How are holdings and profit/loss distributed across assets?
- Which transactions and fees produced these results?

The application must use the supplied data as its source of truth. Do not call a live exchange or market-price API.

Provided Data

`trades.csv`

Approximately 200 synthetic BUY and SELL transactions over six months.

Field	Description
trade_id	Unique transaction identifier
timestamp	UTC ISO-8601 timestamp
exchange	Binance or Coinbase
symbol	BTC, ETH, SOL, CKB, or DOGE
side	BUY or SELL
quantity	Asset quantity traded
price_usd	Execution price per unit in USD
fee_usd	Transaction fee in USD

The history includes repeated buys, partial sells, full closes, reopened positions, and fees. The supplied data never creates a short position.

prices.csv

A current USD price snapshot for each asset.

Field	Description
as_of	UTC timestamp of the snapshot
symbol	Asset symbol
price_usd	Current price per unit in USD

Display the price timestamp in the application so users know when the portfolio was valued.

Calculation Rules

Use weighted-average cost basis. Process transactions in ascending timestamp order for each asset in the calculation scope.

BUY

```
gross buy value = quantity x price_usd
cost added      = gross buy value + fee_usd
new quantity    = previous quantity + bought quantity
new cost basis  = previous cost basis + cost added
average cost    = new cost basis / new quantity
```

The BUY fee is capitalized into cost basis.

SELL

```
gross proceeds    = quantity x price_usd
net proceeds      = gross proceeds - fee_usd
cost removed      = average cost before sale x sold quantity
realized P&L on sale = net proceeds - cost removed
new quantity      = previous quantity - sold quantity
new cost basis     = previous cost basis - cost removed
```

The SELL fee reduces proceeds. A SELL does not change the average cost of the remaining position. If a position is fully closed, set its remaining quantity, cost basis, and average cost to zero before processing a later BUY.

Current Valuation

```
current value      = quantity held x current price
unrealized P&L     = current value - current cost basis
total P&L          = realized P&L + unrealized P&L
allocation         = asset current value / portfolio current value
total fees         = sum of BUY fees and SELL fees
```

Preserve sufficient precision during calculations. Round only for display. Document the numeric type and rounding approach you choose.

Core Requirements

Data Import and Validation

The application must load the supplied files and provide a way to import or re-import `trades.csv`.

Validate at minimum:

- Required columns are present.
- `trade_id` values are unique.
- Timestamps are valid.
- Exchange, symbol, and side values are supported.
- Quantity and price are greater than zero.

- Fees are zero or greater.
- A SELL never exceeds the quantity available at that point in the ordered history.

Show clear, actionable validation errors. An invalid file must not leave the application in a partially imported state.

Dashboard

Display at minimum:

- Current portfolio value
- Current cost basis
- Realized P&L
- Unrealized P&L
- Total P&L
- Total fees paid

Use consistent currency, sign, and percentage formatting. Make positive and negative performance easy to distinguish without relying on color alone.

Holdings Table

Show one row per asset with:

- Quantity held
- Weighted-average cost
- Current price
- Current cost basis
- Current value
- Realized P&L
- Unrealized P&L
- Total P&L
- Portfolio allocation

Closed assets may remain visible if their realized P&L is non-zero.

Charts

Include at least two useful charts. At minimum, show:

- Portfolio allocation by current value
- Realized and unrealized P&L by asset

Charts and headline values must reconcile with the holdings table. Handle zero and negative values clearly.

Transaction Explorer

Provide a transaction table that supports:

- Search or filtering by asset
- Filtering by exchange and side
- Date-range filtering
- Sorting by timestamp
- A practical way to browse all rows, such as pagination or virtualization

Display all fields from `trades.csv`. Show gross trade value and make fees visible.

Empty, Loading, and Error States

Handle first load, missing prices, zero holdings, invalid imports, and unexpected failures. A calculation or import failure must be visible to the user.

Technical Expectations

Any technology stack is acceptable.

The submission should demonstrate:

- A clear frontend/backend boundary or equivalent full-stack architecture

- Deterministic calculation logic separated from presentation code
- Typed or otherwise explicit data contracts
- Sensible component and module boundaries
- Secure handling of configuration and secrets
- Responsive, accessible UI for common desktop and mobile sizes
- Clear error handling and observable failure states

Do not add blockchain integration, authentication, live trading, or live market data unless essential to your design. Explain material assumptions and tradeoffs.

Testing

Add meaningful automated tests for the portfolio calculation and import boundary.

At minimum, cover:

- Multiple BUYs at different prices
- BUY fees included in average cost
- Partial SELLs
- SELL fees deducted from proceeds
- Full close followed by a new BUY
- Rejection of a SELL that would create a short position
- Invalid or duplicate CSV rows

The evaluator should be able to run the test suite with one documented command. Tests must assert known numeric results, not only that code executes.

Deployment Requirements

Deploy the application to a publicly accessible URL. It must work without local setup and remain available for review.

Any hosting platform is acceptable. If authentication is used, provide test credentials. Do not commit secrets to the repository.

Deliverables

Submit:

- Source-code repository
- Live application URL
- README .md
- AI_WORKFLOW .md

Keep the supplied CSV files available to the evaluator. Include clear instructions for loading or resetting the sample data.

README Requirements

The README must cover:

- Local setup and environment variables
- Commands for development, tests, and production build
- Architecture and data flow
- Portfolio calculation approach
- Precision and rounding decisions
- Deployment URL
- Assumptions, limitations, and tradeoffs
- Future improvements

AI Coding-Agent Workflow

Using an AI coding agent is a required part of this assessment. You may use Claude Code, Codex, Cursor, GitHub Copilot, Windsurf, or a comparable tool.

Create an `AI_WORKFLOW.md` with **5-8 meaningful examples** from your actual work. Collectively, the examples must cover:

- Requirement analysis and planning
- Architecture or data-model decisions
- Implementation
- Testing or debugging
- At least one case where you corrected or rejected AI-generated output

For each example, include:

- 1. Goal and context** - what you were trying to achieve.
- 2. Prompt** - the exact prompt or a faithful excerpt with its relevant context, constraints, and acceptance criteria.
- 3. Agent response** - a concise summary of what the agent proposed or changed.
- 4. Your review** - what you checked and how you verified it.
- 5. Outcome** - what you accepted, edited, rejected, or asked the agent to revise, and why.

Do not submit a raw chat export. Select examples that show judgment, iteration, and verification. Links to relevant commits, diffs, tests, or files are encouraged.

State which tools and models you used. If the agent produced most of a file or feature, say so. You own all submitted code and must be able to explain and modify it during review.

Evaluation Criteria

Category	Weight
AI Coding-Agent Workflow	35%
Data & Calculation Correctness	30%
Architecture & Code Quality	15%
Product & UX	10%
Testing & Reliability	5%
Deployment & Documentation	5%

Important Notes

Expected effort is **6-10 hours**.

We value correctness, reasoning, and clear tradeoffs over feature count or visual polish. Keep the solution focused. Do not over-engineer.

AI-generated code is not treated negatively. Unreviewed code, unverifiable calculations, weak prompts, or an inability to explain the submission will be.

Submission

Provide the repository link, deployed application URL, and test credentials if authentication is required.